befoodi

Multi-Restaurant SaaS Platform

Business Account × Multi-Restaurant Tenant Core Architecture Design

Production-Grade SQL Migration & RLS Policy Specification

Document Type	Principal Architect Design Report
Status	**LOCKED BASELINE — Production-Ready**
Platform	befoodi V2 — Multi-Tenant Restaurant SaaS
Architecture	Business Account → Multiple Restaurants (Shared DB / Shared Schema)
Date	February 2026
Authority	Principal SaaS Architect / befoodi Constitutional Baseline
Migration Safety	Additive-only. Zero destructive changes. Production data assumed to exist.

1. Problem Statement

The existing befoodi V2 PVD establishes restaurant_id as the singular tenant pivot in JWT claims and RLS policies. This design assumes a 1:1 relationship between a business owner and their restaurant. The V2 Roadmap now requires a Business Account → Multiple Restaurants architecture, introducing a two-tier tenancy model that the current schema does not support.

The core problem has three dimensions:

- Tenant Identity Ambiguity: A business owner (auth.users entry) may own multiple restaurants across one or more accounts. The JWT cannot carry multiple restaurant_ids simultaneously, creating an identity resolution problem at the RLS enforcement layer.
- Missing Account-Level Isolation Layer: There is currently no accounts table or account_id pivot. Without this, there is no mechanism to group restaurants under a single billing/ownership entity, no way to enforce account-level RLS, and no clean FK chain for account → restaurant → operational data.
- JWT Claim Architecture Conflict: The PVD mandates that tenant identity comes exclusively from server-injected JWT claims. Adding account_id to the JWT requires an Auth Hook redesign. The previous restaurant_id-only JWT claim pattern breaks under multi-restaurant scenarios unless the session carries both account_id and an optionally-scoped restaurant_id.

⛔ **CRITICAL:** This is not a simple additive migration. It introduces a new top-level isolation boundary (account_id) that changes the security model for all downstream tables. Every RLS policy written against restaurant_id alone is now potentially incomplete — it enforces restaurant isolation but does not enforce account isolation.

2. Architectural Risk Register

The following risks were identified through pre-design analysis of the befoodi V2 PVD, the V2 Roadmap, and the proposed Business Account → Multiple Restaurants architecture. Each risk is rated by likelihood and severity. All CRITICAL items must be mitigated before migration deployment.

ID	Risk	Level	Description	Migration Impact
R-01	**Two-Tier JWT Claim Conflict**	HIGH	JWT carries restaurant_id only. Adding account_id creates ambiguity when a user owns restaurants across different accounts. Session must carry active account context or risk policy confusion.	CRITICAL
R-02	**Lookup Loop on account_members**	HIGH	If RLS checks account membership via a subquery to account_members per row, this creates an N+1 pattern at the database level. With 10,000 rows, this fires 10,000 membership checks.	CRITICAL
R-03	**Ghost Restaurants Post Soft-Delete**	MEDIUM	If an account is soft-deleted without cascading to restaurants, those restaurants remain active in RLS context. Any JWT with a valid restaurant_id from a deleted account will still pass restaurant-level policies.	HIGH
R-04	**account_id Missing from Restaurant FK Chain**	HIGH	All downstream tables (orders, menus, staff) FK to restaurant_id. If restaurants table lacks account_id or it's not indexed, account-level queries require a join through restaurants, killing RLS performance.	CRITICAL
R-05	**Race Condition on Account Creation**	MEDIUM	If account creation and account_members insertion are not atomic (single transaction), a partial failure leaves orphaned auth.users with no account membership, bypassing all member-scoped RLS.	HIGH
R-06	**Platform Admin Policy Gap**	MEDIUM	Platform admins have no account_id in JWT. If SELECT policies on accounts only check account_id from JWT, platform admins get zero rows — correct for data isolation but blocks legitimate lifecycle management.	HIGH
R-07	**UNIQUE Constraint on account_members with Soft Delete**	MEDIUM	A hard UNIQUE(account_id, user_id) constraint breaks if a user is removed and re-invited to the same account. Must use partial unique index on deleted_at IS NULL.	MEDIUM
R-08	**slug Uniqueness Scope**	LOW	If account slugs are globally unique, two businesses cannot share similar names. If scoped to platform only, subdomain routing is simple. Must decide before creating the constraint.	MEDIUM
R-09	**Migration Conflict with Existing restaurants Table**	HIGH	Production likely has a restaurants table. Adding account_id as NOT NULL FK is destructive to existing rows. Must add as nullable first, backfill, then add NOT NULL constraint in a separate migration.	CRITICAL
R-10	**Supabase Realtime channel isolation**	LOW	Realtime subscriptions filter via RLS. If account_id is added to restaurants but downstream tables still use restaurant_id-only RLS, Realtime leaks are possible during the transition window.	MEDIUM

3. Architectural Decisions

3.1 Two-Tier Tenancy Model

Decision: Introduce public.accounts as the top-level tenant entity. public.restaurants FK to accounts.id. All downstream tables retain restaurant_id as the operational pivot. Account-level isolation is enforced at the accounts and account_members layer only.

Rationale: Downstream tables (orders, menus, staff) operate within a restaurant context. Rewriting all downstream RLS to use account_id would require touching every existing policy — a massive, risky migration. Instead, restaurant_id remains the operational security boundary while account_id governs ownership and billing. This is the minimal-impact, maximum-safety approach.

3.2 JWT Claim Strategy: Dual Claim Injection

Decision: The Supabase Auth Hook will inject both account_id and restaurant_id into the JWT. restaurant_id may be null when the user context is account-level (e.g., onboarding a new restaurant, viewing billing). restaurant_id is set when the user selects an active restaurant in the session.

> ℹ️ **DECISION:** account_id is the primary isolation boundary for the accounts/account_members/restaurants tables. restaurant_id remains the operational boundary for orders/menus/staff/KDS. Both are server-injected. Neither is trusted from client input.

3.3 Membership Check Strategy: JWT-First, No Lookup Loop

Decision: RLS on accounts and restaurants uses account_id from JWT claims exclusively — never a subquery into account_members. The account_members table is used only by the Auth Hook (Edge Function, service_role) to build the JWT at login time. Once the JWT is issued, account_members is never queried by RLS policies.

Justification: This eliminates the Lookup Loop anti-pattern. account_members may have thousands of rows at scale. A per-row subquery check in RLS would multiply latency by the number of rows evaluated. JWT-based caching with initPlan wrapping is the only pattern that meets the <50ms RLS SLA.

3.4 Migration Safety for Existing restaurants Table

Decision: If a restaurants table already exists in production, account_id will be added as a nullable UUID column first. A backfill strategy (via Edge Function or manual script with service_role) populates existing rows before a second migration enforces NOT NULL. This is a two-migration approach to prevent destructive schema changes.

> ⚠️ **NON-NEGOTIABLE:** This migration MUST NOT use ALTER TABLE restaurants ADD COLUMN account_id UUID NOT NULL in a single step if production rows exist. That statement will fail with a constraint violation. The migration is structured to be safe for both empty and non-empty production tables.

3.5 Soft Delete Pattern

Decision: Both accounts and restaurants use deleted_at TIMESTAMPTZ (not is_deleted boolean). Cascade soft-delete is enforced via database trigger: when an account is soft-deleted, all its restaurants are soft-deleted automatically. This prevents Ghost Restaurant records from persisting in active JWT contexts.

3.6 Account Status State Machine

The account status follows a constrained state machine enforced by a CHECK constraint. Transitions are: pending → active → suspended → closed. The closed state is terminal — no transition out is permitted. This prevents reactivation of closed accounts without an explicit migration.

From	To	Permitted	Trigger
pending	active	✅ Yes	Admin activation + activation code
active	suspended	✅ Yes	Policy violation or manual admin
suspended	active	✅ Yes	Admin reinstatement only
active	closed	✅ Yes	Account closure request (aal2)
closed	any	⛔ No	CHECK constraint blocks transition

4. Final Schema Design

Three tables constitute the Business Account → Restaurant tenant core layer. All tables are in the public schema for PostgREST compatibility. A dedicated audit schema handles append-only compliance logging.

4.1 Table: public.accounts

The top-level tenant entity. Represents a business (e.g., 'Spice Garden Restaurant Group'). One account owns one or more restaurants.

Column	Type	Constraint	Justification
id	UUID	PK, DEFAULT gen_random_uuid()	gen_random_uuid() preferred over uuid_generate_v4() — no extension dependency required in Supabase PG15+.
owner_id	UUID	NOT NULL, FK auth.users	The auth.users record that created this account. ON DELETE RESTRICT prevents orphaned accounts if the owner's auth record is deleted.
name	TEXT	NOT NULL, CHECK length >= 2	Business display name. No uniqueness constraint globally — two businesses can share names. Uniqueness enforced by slug.
slug	TEXT	UNIQUE (partial, active only), NOT NULL	URL-safe identifier used for subdomain routing. Partial unique index on deleted_at IS NULL prevents reuse of active slugs but allows recovery of closed account slugs.
status	TEXT	NOT NULL, CHECK IN (pending, active, suspended, closed)	State machine column. CHECK constraint is the terminal enforcement. Using TEXT not ENUM — ENUMs require DDL to add new values, creating migration risk.
plan_tier	TEXT	NOT NULL, DEFAULT 'starter'	Billing tier. Used downstream for feature-gating logic. TEXT not ENUM for the same migration flexibility reason.
settings	JSONB	DEFAULT '{}'	JUSTIFIED JSON usage: account-level settings are sparse, varied per plan tier, and not queried in RLS predicates. GIN index applied. Never used as a security boundary.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	TIMESTAMPTZ (not TIMESTAMP) — stores timezone offset, critical for multi-region deployments and 7-10y audit retention.
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Updated by trigger on every row mutation. Enables incremental sync and drift detection.
deleted_at	TIMESTAMPTZ	NULL	Soft delete. NULL = active. Timestamp = deactivated. Cascading trigger fires on all child restaurants when this is set.

4.1.1 Index Strategy — public.accounts

Index Name	Type	Justification
accounts_pkey	B-TREE (PK)	Default PK index. No action needed.

Index Name	Type	Justification
`idx_accounts_owner_id`	B-TREE	Auth Hook lookup: 'find accounts where owner_id = auth.uid()'. Fires on every login. Without this index, full table scan on accounts per login.
`uidx_accounts_slug_active`	B-TREE PARTIAL	Partial unique index: UNIQUE WHERE deleted_at IS NULL. Prevents duplicate active slugs while allowing reuse of closed account slugs.
`idx_accounts_status`	B-TREE	Platform admin queries filter by status (e.g., 'all pending accounts'). Without index, full scan at scale.
`idx_accounts_deleted_at`	B-TREE PARTIAL	Partial index WHERE deleted_at IS NULL. All active-account queries use this. Eliminates soft-deleted rows from most scans.

4.2 Table: public.account_members

Join table linking auth.users to accounts with a scoped role. This table is read by the Auth Hook to build JWT claims. It is NOT queried by RLS policies at runtime — JWT claims are the runtime enforcement mechanism.

Column	Type	Constraint	Justification
`id`	`UUID`	PK	Surrogate key. Required because a composite PK on (account_id, user_id) would make re-invitation after soft-delete impossible without violating PK constraints.
`account_id`	`UUID`	NOT NULL, FK accounts(id)	ON DELETE CASCADE: if an account is hard-deleted (emergency only), memberships cascade. Soft-delete is the normal path and is handled by trigger.
`user_id`	`UUID`	NOT NULL, FK auth.users(id)	ON DELETE CASCADE: if an auth.users record is deleted, their memberships are automatically removed. Prevents ghost memberships.
`role`	`TEXT`	NOT NULL, CHECK IN (owner, admin, member)	Account-level RBAC. 'owner' is set at account creation and maps to full management rights. 'admin' can manage restaurants but not billing. 'member' is read-only at account level.
`invited_by`	`UUID`	FK auth.users(id) , NULL	Audit trail for team invitations. NULL for the initial owner. ON DELETE SET NULL — audit trail preserved even if inviter's account is removed.
`joined_at`	`TIMESTAMPTZ`	NOT NULL, DEFAULT now()	Immutable after insert. Records when membership became active.
`deleted_at`	`TIMESTAMPTZ`	NULL	Soft delete for membership removal. Preserves historical audit trail of who was a member and when they were removed.

⚠️ **CONSTRAINT WARNING:** A hard UNIQUE(account_id, user_id) is BANNED. Use partial unique index uidx_account_members_active on (account_id, user_id) WHERE deleted_at IS NULL. This allows re-invitation of previously removed members without constraint violations.

4.3 Table: public.restaurants

The operational tenant entity. FKs to accounts. All downstream tables (orders, menus, staff) reference this table via restaurant_id. The restaurant_id is the JWT claim used for operational RLS.

Column	Type	Constraint	Justification
id	UUID	PK, DEFAULT gen_random_uuid()	Operational tenant pivot. This UUID becomes the restaurant_id JWT claim for all downstream RLS.
account_id	UUID	NOT NULL*, FK accounts(id)	*MIGRATION CRITICAL: Added as nullable in migration step 1, backfilled in step 2, NOT NULL constraint added in step 3. ON DELETE RESTRICT — cannot delete an account that has restaurants.
name	TEXT	NOT NULL, CHECK length >= 2	Restaurant display name. Scoped uniqueness within account enforced by partial index, not global uniqueness.
slug	TEXT	NOT NULL, UNIQUE within account	Partial unique index: UNIQUE(account_id, slug) WHERE deleted_at IS NULL. A restaurant group can have 'downtown' and 'uptown' slugs but not two 'downtown' in the same account.
status	TEXT	NOT NULL, CHECK, DEFAULT 'pending'	Same state machine as accounts. Status is independent — a restaurant can be suspended while its parent account is active.
address_line1	TEXT	NULL	Physical address. NULL allowed during onboarding before activation. Not in RLS predicates — no index required.
city	TEXT	NULL	City for display and regional reporting. No index — not a filter column.
timezone	TEXT	NOT NULL, DEFAULT 'UTC'	IANA timezone identifier (e.g., 'Asia/Kolkata'). Required for rush-hour scheduling, deployment window enforcement, and report generation.
max_tables	SMALLINT	NOT NULL, DEFAULT 20, CHECK > 0	Maximum concurrent table sessions. SMALLINT sufficient (range 1–32767). Enforced by trigger when creating new table sessions.
settings	JSONB	DEFAULT '{}'	Restaurant-level settings (KDS display preferences, menu sort, tax config). JSONB JUSTIFIED: sparse, not in RLS predicates, GIN indexed.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Immutable creation timestamp. Used in audit logs and lifecycle reporting.
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Maintained by trigger. Enables incremental sync for read replicas.
deleted_at	TIMESTAMPTZ	NULL	Soft delete. Cascade trigger fires when parent account is soft-deleted.

5. Production-Ready SQL Migration

> ⚠️ **MIGRATION ORDER:** Migration 20260217000000 is safe to run on production with existing data. Migration 20260217000001 MUST be run only after the backfill process assigns account_id to all existing restaurants rows. Attempting to run it before backfill will throw a RAISE EXCEPTION and abort safely.

5.1 Migration 20260217000000 — Tenant Core Tables

```sql
-- ============================================================
-- MIGRATION: 20260217000000_business_account_tenant_core.sql
-- befoodi V2 — Business Account Multi-Restaurant Tenant Core
-- SAFETY CLASS: Additive-only. Zero destructive operations.
-- ROLLBACK: See rollback script section.
-- PRE-CONDITION: Run on staging with production clone first.
-- ============================================================

-- ─── GUARD: Prevent accidental re-execution ────────────────
DO $$ BEGIN
  IF EXISTS (SELECT 1 FROM information_schema.tables
    WHERE table_schema = 'public' AND table_name = 'accounts')
  THEN RAISE NOTICE 'accounts table exists, skipping DDL';
  END IF;
END $$;

-- ─── STEP 1: Create public.accounts ────────────────────────
CREATE TABLE IF NOT EXISTS public.accounts (
  id          UUID        NOT NULL DEFAULT gen_random_uuid(),
  owner_id    UUID        NOT NULL
                REFERENCES auth.users(id) ON DELETE RESTRICT
                DEFERRABLE INITIALLY DEFERRED,
  name        TEXT        NOT NULL
                CONSTRAINT accounts_name_min_length
                CHECK (char_length(name) >= 2),
  slug        TEXT        NOT NULL
                CONSTRAINT accounts_slug_format
                CHECK (slug ~ '^[a-z0-9][a-z0-9\-]{1,61}[a-z0-9]$'),
  status      TEXT        NOT NULL DEFAULT 'pending'
                CONSTRAINT accounts_status_valid
                CHECK (status IN ('pending','active','suspended','closed')),
  plan_tier   TEXT        NOT NULL DEFAULT 'starter'
                CONSTRAINT accounts_plan_tier_valid
                CHECK (plan_tier IN ('starter','growth','enterprise')),
  settings    JSONB       NOT NULL DEFAULT '{}',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at  TIMESTAMPTZ,
  CONSTRAINT  accounts_pkey PRIMARY KEY (id)
);

-- ─── STEP 2: Indexes for public.accounts ───────────────────
CREATE INDEX IF NOT EXISTS idx_accounts_owner_id
  ON public.accounts(owner_id);

CREATE INDEX IF NOT EXISTS idx_accounts_status
  ON public.accounts(status)
  WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_accounts_deleted_at
  ON public.accounts(deleted_at)
  WHERE deleted_at IS NULL;

-- Partial unique: active slugs must be unique
CREATE UNIQUE INDEX IF NOT EXISTS uidx_accounts_slug_active
  ON public.accounts(slug)
  WHERE deleted_at IS NULL;
```

```sql
CREATE INDEX IF NOT EXISTS idx_accounts_settings_gin
  ON public.accounts USING GIN (settings);

-- ─── STEP 3: Create public.account_members ─────────────────
CREATE TABLE IF NOT EXISTS public.account_members (
  id          UUID        NOT NULL DEFAULT gen_random_uuid(),
  account_id  UUID        NOT NULL
                REFERENCES public.accounts(id) ON DELETE CASCADE
                DEFERRABLE INITIALLY DEFERRED,
  user_id     UUID        NOT NULL
                REFERENCES auth.users(id) ON DELETE CASCADE
                DEFERRABLE INITIALLY DEFERRED,
  role        TEXT        NOT NULL DEFAULT 'member'
                CONSTRAINT account_members_role_valid
                CHECK (role IN ('owner','admin','member')),
  invited_by  UUID
                REFERENCES auth.users(id) ON DELETE SET NULL
                DEFERRABLE INITIALLY DEFERRED,
  joined_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at  TIMESTAMPTZ,
  CONSTRAINT  account_members_pkey PRIMARY KEY (id)
);

-- ─── STEP 4: Indexes for public.account_members ────────────
-- Partial unique: one active membership per user per account
CREATE UNIQUE INDEX IF NOT EXISTS uidx_account_members_active
  ON public.account_members(account_id, user_id)
  WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_account_members_user_id
  ON public.account_members(user_id)
  WHERE deleted_at IS NULL;

CREATE INDEX IF NOT EXISTS idx_account_members_account_id
  ON public.account_members(account_id)
  WHERE deleted_at IS NULL;

-- ─── STEP 5: Add account_id to restaurants (SAFE) ──────────
-- NOTE: Added as NULLABLE first. NOT NULL enforced in
--       migration 20260217000001 after backfill.
ALTER TABLE public.restaurants
  ADD COLUMN IF NOT EXISTS account_id UUID
    REFERENCES public.accounts(id) ON DELETE RESTRICT
    DEFERRABLE INITIALLY DEFERRED;

-- ─── STEP 6: Indexes for restaurants.account_id ────────────
CREATE INDEX IF NOT EXISTS idx_restaurants_account_id
  ON public.restaurants(account_id)
  WHERE deleted_at IS NULL;

-- ─── STEP 7: updated_at trigger function (idempotent) ──────
CREATE OR REPLACE FUNCTION public.fn_set_updated_at()
  RETURNS TRIGGER
  LANGUAGE plpgsql
  SECURITY INVOKER
  STABLE
AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$;

-- ─── STEP 8: Attach updated_at triggers ────────────────────
DROP TRIGGER IF EXISTS trg_accounts_updated_at ON public.accounts;
CREATE TRIGGER trg_accounts_updated_at
  BEFORE UPDATE ON public.accounts
  FOR EACH ROW EXECUTE FUNCTION public.fn_set_updated_at();

-- ─── STEP 9: Cascade soft-delete trigger ───────────────────
CREATE OR REPLACE FUNCTION public.fn_cascade_account_soft_delete()
  RETURNS TRIGGER
```

```
  LANGUAGE plpgsql
  SECURITY DEFINER
  STABLE
AS $$
BEGIN
  IF NEW.deleted_at IS NOT NULL AND OLD.deleted_at IS NULL THEN
    -- Cascade soft delete to all member records
    UPDATE public.account_members
      SET deleted_at = NEW.deleted_at
      WHERE account_id = NEW.id AND deleted_at IS NULL;
    -- Cascade soft delete to all restaurants
    UPDATE public.restaurants
      SET deleted_at = NEW.deleted_at
      WHERE account_id = NEW.id AND deleted_at IS NULL;
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_accounts_cascade_soft_delete
  ON public.accounts;
CREATE TRIGGER trg_accounts_cascade_soft_delete
  AFTER UPDATE ON public.accounts
  FOR EACH ROW EXECUTE FUNCTION public.fn_cascade_account_soft_delete();

-- ─── STEP 10: Enable RLS ──────────────────────────────────────
ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.account_members ENABLE ROW LEVEL SECURITY;
-- Note: restaurants RLS assumed already enabled per PVD.
-- Idempotent: enable is safe if already enabled.

-- ─── STEP 11: Revoke public access ────────────────────────────
REVOKE ALL ON public.accounts FROM PUBLIC;
REVOKE ALL ON public.account_members FROM PUBLIC;
GRANT SELECT, INSERT, UPDATE ON public.accounts TO authenticated;
GRANT SELECT, INSERT, UPDATE ON public.account_members TO authenticated;

-- End of migration 20260217000000
```

5.2 Migration 20260217000001 — NOT NULL Enforcement (Post-Backfill)

```
SQL
-- ============================================================
-- MIGRATION: 20260217000001_restaurants_account_id_not_null.sql
-- PURPOSE: Enforce NOT NULL on restaurants.account_id after
--          backfill. Run ONLY after all existing restaurants
--          have been assigned an account_id via Edge Function.
-- PRE-CONDITION: SELECT COUNT(*) FROM restaurants
--                WHERE account_id IS NULL must return 0.
-- ============================================================

DO $$ BEGIN
  IF EXISTS (
    SELECT 1 FROM public.restaurants WHERE account_id IS NULL
  ) THEN
    RAISE EXCEPTION
      'ABORT: % restaurants have NULL account_id. Backfill first.'
      , (SELECT COUNT(*) FROM public.restaurants
           WHERE account_id IS NULL);
  END IF;
END $$;

ALTER TABLE public.restaurants
  ALTER COLUMN account_id SET NOT NULL;

-- Add composite unique: restaurant slugs unique within account
CREATE UNIQUE INDEX IF NOT EXISTS uidx_restaurants_slug_account
  ON public.restaurants(account_id, slug)
  WHERE deleted_at IS NULL;
```

```
-- End of migration 20260217000001
```

6. RLS Policy Specification

⛔ **CRITICAL PATTERN:** All RLS policies use the (SELECT auth.jwt() ->> 'claim')::type pattern. The SELECT wrapper forces PostgreSQL to use initPlan, evaluating the JWT extraction ONCE per query and caching it for the duration. Without this wrapper, JWT extraction fires for every row evaluated, multiplying latency by row count.

6.1 RLS Policy Summary

Table	Op	USING Condition	WITH CHECK Condition	Security Rationale
`accounts`	SELECT	JWT account_id match	OR platform_admin claim	Account members + Platform Admins
`accounts`	INSERT	Blocked unless owner_id = auth.uid()	status must be 'pending'	Authenticated users only; prevents impersonation
`accounts`	UPDATE	JWT account_id match + role IN (owner, admin)	Cannot change owner_id or update closed account	Owners and Admins only; terminal state enforced
`accounts`	DELETE	USING (false)	Hard deletes blocked for all authenticated users	Soft-delete path only; service_role Edge Function exception
`account_members`	SELECT	JWT account_id match + active membership	—	Members see their own account roster only
`account_members`	INSERT	JWT account_id + role IN (owner, admin)	invited_by = auth.uid() enforced	Prevents member self-invitation or cross-account injection
`account_members`	UPDATE	JWT account_id + role = owner only	Cannot set role to 'owner' via UPDATE	Prevents admin-to-owner self-escalation
`account_members`	DELETE	USING (false)	Soft-delete only	Preserves membership audit history
`restaurants`	SELECT	JWT account_id match	OR platform_admin claim	Account-scoped; replaces any 'global SELECT' policy
`restaurants`	INSERT	JWT account_id + role IN (owner, admin)	status = 'pending' enforced on create	Prevents rogue restaurant creation under wrong account
`restaurants`	UPDATE	JWT account_id + role IN (owner, admin)	account_id immutable in WITH CHECK	Cannot move restaurant between accounts via UPDATE
`restaurants`	DELETE	USING (false)	Hard deletes blocked	Cascade soft-delete from accounts trigger only

6.2 public.accounts RLS Policies

```sql
-- ─── DROP EXISTING POLICIES (idempotent) ──────────────────
DROP POLICY IF EXISTS "accounts_select_own" ON public.accounts;
DROP POLICY IF EXISTS "accounts_select_platform_admin"
  ON public.accounts;
DROP POLICY IF EXISTS "accounts_insert_authenticated"
  ON public.accounts;
```

```sql
DROP POLICY IF EXISTS "accounts_update_owner" ON public.accounts;
DROP POLICY IF EXISTS "accounts_delete_blocked" ON public.accounts;

-- ─── POLICY 1: SELECT — account members see their account ──
-- Uses JWT claim 'account_id' injected by Auth Hook.
-- initPlan caching: SELECT wrapper evaluated ONCE per query.
CREATE POLICY "accounts_select_own"
  ON public.accounts
  FOR SELECT
  TO authenticated
  USING (
    id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND deleted_at IS NULL
  );

-- ─── POLICY 2: SELECT — platform admin sees lifecycle data —
-- Platform admin JWT has is_platform_admin: true, no account_id
CREATE POLICY "accounts_select_platform_admin"
  ON public.accounts
  FOR SELECT
  TO authenticated
  USING (
    (SELECT (auth.jwt() ->> 'is_platform_admin')::boolean) = true
  );

-- ─── POLICY 3: INSERT — authenticated users create accounts —
-- WITH CHECK enforces owner_id = auth.uid(). Client cannot
-- forge a different owner. Status locked to 'pending'.
CREATE POLICY "accounts_insert_authenticated"
  ON public.accounts
  FOR INSERT
  TO authenticated
  WITH CHECK (
    owner_id = auth.uid()
    AND status = 'pending'
  );

-- ─── POLICY 4: UPDATE — account owner or admin only ─────────
-- Role check via JWT claim 'account_role'. Prevents members
-- from modifying account settings or status.
CREATE POLICY "accounts_update_owner"
  ON public.accounts
  FOR UPDATE
  TO authenticated
  USING (
    id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND (SELECT auth.jwt() ->> 'account_role') IN ('owner','admin')
    AND deleted_at IS NULL
  )
  WITH CHECK (
    id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND owner_id = owner_id  -- immutable: cannot transfer ownership via UPDATE
    AND status != 'closed'   -- terminal state: cannot update closed accounts
  );

-- ─── POLICY 5: DELETE — hard deletes blocked for all ────────
-- All deletions must go through soft-delete (deleted_at).
-- Hard DELETE is denied at the RLS layer for all roles.
-- Only AAL2 Edge Functions with service_role bypass this.
CREATE POLICY "accounts_delete_blocked"
  ON public.accounts
  FOR DELETE
  TO authenticated
  USING (false);
```

6.3 public.account_members RLS Policies

```sql
DROP POLICY IF EXISTS "account_members_select_own"
  ON public.account_members;
DROP POLICY IF EXISTS "account_members_insert_owner"
  ON public.account_members;
```

```
DROP POLICY IF EXISTS "account_members_update_owner"
  ON public.account_members;
DROP POLICY IF EXISTS "account_members_delete_blocked"
  ON public.account_members;

-- ─── POLICY 1: SELECT — members see their account's roster —
CREATE POLICY "account_members_select_own"
  ON public.account_members
  FOR SELECT
  TO authenticated
  USING (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND deleted_at IS NULL
  );

-- ─── POLICY 2: INSERT — account owner/admin invites members —
CREATE POLICY "account_members_insert_owner"
  ON public.account_members
  FOR INSERT
  TO authenticated
  WITH CHECK (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND (SELECT auth.jwt() ->> 'account_role') IN ('owner','admin')
    AND invited_by = auth.uid()
  );

-- ─── POLICY 3: UPDATE — role changes by owner only ──────────
-- Admins cannot promote themselves to owner.
CREATE POLICY "account_members_update_owner"
  ON public.account_members
  FOR UPDATE
  TO authenticated
  USING (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND (SELECT auth.jwt() ->> 'account_role') = 'owner'
    AND deleted_at IS NULL
  )
  WITH CHECK (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    -- Prevent role escalation to 'owner' via UPDATE
    AND role != 'owner'
  );

-- ─── POLICY 4: DELETE — hard deletes blocked ────────────────
CREATE POLICY "account_members_delete_blocked"
  ON public.account_members
  FOR DELETE
  TO authenticated
  USING (false);
```

6.4 public.restaurants — Additive Account-Level Policies

```
SQL
-- ADDITIVE POLICIES on existing restaurants table
-- Adds account-level isolation without removing existing
-- restaurant_id policies.

DROP POLICY IF EXISTS "restaurants_select_account"
  ON public.restaurants;
DROP POLICY IF EXISTS "restaurants_insert_account"
  ON public.restaurants;
DROP POLICY IF EXISTS "restaurants_update_account"
  ON public.restaurants;

-- ─── POLICY: SELECT — account members see their restaurants —
-- Checks account_id from JWT. Replaces any 'see all' policy.
CREATE POLICY "restaurants_select_account"
  ON public.restaurants
  FOR SELECT
  TO authenticated
  USING (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
```

```
    AND deleted_at IS NULL
  );

-- ─── POLICY: INSERT — account owner/admin creates restaurants
CREATE POLICY "restaurants_insert_account"
  ON public.restaurants
  FOR INSERT
  TO authenticated
  WITH CHECK (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND (SELECT auth.jwt() ->> 'account_role') IN ('owner','admin')
    AND status = 'pending'
  );

-- ─── POLICY: UPDATE — scoped to account + restaurant_id ─────
-- When restaurant_id is in JWT (restaurant-scoped session),
-- checks both account_id AND restaurant_id for dual isolation.
CREATE POLICY "restaurants_update_account"
  ON public.restaurants
  FOR UPDATE
  TO authenticated
  USING (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
    AND (SELECT auth.jwt() ->> 'account_role') IN ('owner','admin')
    AND deleted_at IS NULL
  )
  WITH CHECK (
    account_id = (SELECT (auth.jwt() ->> 'account_id')::uuid)
  );
```

7. Safety Analysis

7.1 Why Alternative Designs Are Dangerous

Alternative A: Using account_members in RLS USING clause

⛔ **REJECTED:** CREATE POLICY 'x' ON accounts FOR SELECT USING (EXISTS (SELECT 1 FROM account_members WHERE user_id = auth.uid() AND account_id = accounts.id)). This fires a correlated subquery for every row in accounts. At 1000 accounts × 10 rows returned, that's 10,000 membership checks. This violates the <50ms RLS SLA and introduces the Lookup Loop anti-pattern explicitly banned in the PVD.

Alternative B: Storing account_id in user_metadata

⛔ **REJECTED:** user_metadata is a client-writable field in Supabase Auth. A malicious user can POST to /auth/v1/user and set user_metadata.account_id to any UUID, bypassing tenant isolation entirely. The PVD Section 2.2 explicitly bans storing sensitive tokens in user_metadata.

Alternative C: USING (true) for platform admin bypass

⛔ **REJECTED:** USING (true) grants read access to all rows regardless of tenant. This is banned by the PVD and allows platform admins to see restaurant operational data, customer PII, and financial records — violating Section 7 Admin Isolation Rules.

Alternative D: Single migration adding account_id NOT NULL immediately

⛔ **REJECTED:** ALTER TABLE restaurants ADD COLUMN account_id UUID NOT NULL will fail with 'column contains null values' if any rows exist in production. This is a destructive migration that causes immediate production downtime. The two-migration pattern (nullable → backfill → NOT NULL) is the only production-safe approach.

7.2 What Would Break Multi-Tenancy

Breach Scenario	Root Cause	Prevention in This Design
Account A user reads Account B restaurants	RLS on restaurants checks restaurant_id only, not account_id	restaurants_select_account policy enforces account_id match. restaurant_id alone is insufficient after this migration.
User assigns restaurant to wrong account	INSERT policy on restaurants does not verify account_id against JWT	WITH CHECK on restaurants INSERT enforces account_id = JWT claim. Client-provided account_id is rejected if it doesn't match.
Soft-deleted account's restaurants remain accessible	No cascade trigger from account to restaurants on soft delete	fn_cascade_account_soft_delete trigger fires on account deleted_at SET, cascades to restaurants and account_members atomically.
Member escalates own role to owner	UPDATE policy on account_members allows any role value in WITH CHECK	WITH CHECK on account_members UPDATE enforces role != 'owner'. Only initial INSERT at account creation can set owner role.

Breach Scenario	Root Cause	Prevention in This Design
JWT forgery bypasses account_id check	JWT signed with wrong secret or client-side JWT manipulation	Supabase JWT signed with environment-specific secret. auth.jwt() is server-side extraction. Client cannot forge claims without the signing key.

7.3 What Could Break Production CI

- Missing backfill before migration 20260217000001: The DO $$ BEGIN guard detects NULL account_id rows and throws RAISE EXCEPTION, aborting the migration safely. CI build fails rather than silently applying a partial schema.
- Duplicate slug on accounts table: The uidx_accounts_slug_active partial unique index enforces uniqueness only on active records. Test data with duplicate slugs in staging will surface this as a constraint violation before production deploy.
- Existing restaurants table without account_id: migration 20260217000000 uses ADD COLUMN IF NOT EXISTS, which is idempotent. If account_id was already added manually (a governance failure), the migration skips the ADD COLUMN safely.
- RLS policies on restaurants already existing with conflicting names: The DROP POLICY IF EXISTS pattern before each CREATE POLICY ensures clean idempotent application. No duplicate policy name conflicts.
- auth.users FK violation: DEFERRABLE INITIALLY DEFERRED allows batch inserts within a transaction where the auth.users record is created first, then the accounts record. Without DEFERRABLE, FK violations fire before the transaction commits.

7.4 Rollback Script

⚠️ **ROLLBACK POLICY:** The rollback script must be tested in staging before production deployment. Rollback of migration 20260217000001 requires careful handling — if NOT NULL was enforced, rollback must drop that constraint, not the column. Dropping account_id from restaurants is DESTRUCTIVE and is NOT in the rollback script.

```sql
-- ROLLBACK: 20260217000000_business_account_tenant_core.sql
-- SAFE TO RUN if: no downstream data depends on accounts table
-- DESTRUCTIVE if: restaurants.account_id has been backfilled

-- STEP 1: Remove RLS policies
DROP POLICY IF EXISTS "accounts_select_own" ON public.accounts;
DROP POLICY IF EXISTS "accounts_select_platform_admin"
  ON public.accounts;
DROP POLICY IF EXISTS "accounts_insert_authenticated"
  ON public.accounts;
DROP POLICY IF EXISTS "accounts_update_owner" ON public.accounts;
DROP POLICY IF EXISTS "accounts_delete_blocked" ON public.accounts;
DROP POLICY IF EXISTS "account_members_select_own"
  ON public.account_members;
DROP POLICY IF EXISTS "account_members_insert_owner"
  ON public.account_members;
DROP POLICY IF EXISTS "account_members_update_owner"
  ON public.account_members;
DROP POLICY IF EXISTS "account_members_delete_blocked"
  ON public.account_members;
DROP POLICY IF EXISTS "restaurants_select_account"
  ON public.restaurants;
DROP POLICY IF EXISTS "restaurants_insert_account"
  ON public.restaurants;
DROP POLICY IF EXISTS "restaurants_update_account"
  ON public.restaurants;

-- STEP 2: Drop triggers
```

```
DROP TRIGGER IF EXISTS trg_accounts_cascade_soft_delete
  ON public.accounts;
DROP TRIGGER IF EXISTS trg_accounts_updated_at ON public.accounts;

-- STEP 3: Drop functions
DROP FUNCTION IF EXISTS public.fn_cascade_account_soft_delete();

-- STEP 4: Drop indexes on restaurants
DROP INDEX IF EXISTS idx_restaurants_account_id;

-- STEP 5: Remove account_id from restaurants (ONLY if not backfilled)
-- WARNING: VERIFY BEFORE RUNNING
-- ALTER TABLE public.restaurants DROP COLUMN IF EXISTS account_id;

-- STEP 6: Drop account_members table
DROP TABLE IF EXISTS public.account_members;

-- STEP 7: Drop accounts table
DROP TABLE IF EXISTS public.accounts;
```

8. Auth Hook: JWT Claim Injection Strategy

The Supabase Auth Hook is the only place where account_id, account_role, and restaurant_id are injected into the JWT. This Edge Function runs on every authentication event. It uses service_role to query account_members — the one permitted use of the service_role key outside of client-facing APIs.

8.1 JWT Claim Structure — Business Account Model

JWT Claim	Type	Source	RLS Usage
`account_id`	UUID	account_members	Primary isolation pivot for accounts, account_members, restaurants tables.
`account_role`	TEXT	account_members	Used in UPDATE/INSERT policies to restrict management operations to owner/admin.
`restaurant_id`	UUID | null	Session selection	Operational pivot for orders, menus, staff. NULL when session is account-scoped.
`app_role`	TEXT	User type	admin | staff | customer. Used in restaurant-level operational RLS.
`is_platform_admin`	BOOLEAN	Platform admin flag	Grants access to accounts SELECT (lifecycle data only). Explicitly excludes restaurant operational data.

ℹ️ **SECURITY MANDATE:** The Auth Hook Edge Function is the ONLY code that may query account_members using service_role. All other queries to account_members must go through RLS-enforced authenticated client access. The Auth Hook result is cryptographically signed in the JWT — clients cannot modify claim values.

9. pgTAP Negative Test Suite

Per the befoodi PVD, comprehensive negative test cases must prove that cross-tenant data access is impossible under all conditions. The following tests must pass before the migration is promoted to production. They test ABSENCE of data — zero rows returned or exceptions thrown — which proves isolation.

```sql
-- pgTAP Negative Test Suite: Business Account Tenant Isolation
-- Run via: supabase test db

BEGIN;
SELECT plan(8);

-- TEST 1: Account A user cannot SELECT Account B accounts
SET LOCAL jwt.claims.account_id =
  '00000000-0000-0000-0000-000000000001';
SELECT is(
  (SELECT COUNT(*)::int FROM public.accounts
   WHERE id = '00000000-0000-0000-0000-000000000002'),
  0,
  'Account A cannot read Account B records'
);

-- TEST 2: Cannot INSERT account with forged owner_id
-- Attempt to create account owned by different user
SELECT throws_ok(
  $$ INSERT INTO public.accounts(owner_id, name, slug)
     VALUES('aaaaaaaa-0000-0000-0000-000000000001',
            'Forged Account', 'forged-slug') $$,
  'new row violates row-level security',
  'Cannot INSERT account with different owner_id'
);

-- TEST 3: member role cannot UPDATE account
SET LOCAL jwt.claims.account_role = 'member';
SELECT throws_ok(
  $$ UPDATE public.accounts SET name = 'Hacked'
     WHERE id = '00000000-0000-0000-0000-000000000001' $$,
  'new row violates row-level security',
  'Member role cannot UPDATE account'
);

-- TEST 4: Hard DELETE on accounts blocked
SELECT throws_ok(
  $$ DELETE FROM public.accounts
     WHERE id = '00000000-0000-0000-0000-000000000001' $$,
  'new row violates row-level security',
  'Hard DELETE on accounts is blocked by RLS'
);

-- TEST 5: Cannot self-escalate to owner via account_members UPDATE
SET LOCAL jwt.claims.account_role = 'admin';
SELECT throws_ok(
  $$ UPDATE public.account_members SET role = 'owner'
     WHERE user_id = auth.uid() $$,
  'new row violates row-level security',
  'Admin cannot self-escalate to owner'
);

-- TEST 6: Soft-deleted account restaurants not visible
-- (account deleted_at IS NOT NULL)
SELECT is(
  (SELECT COUNT(*)::int FROM public.restaurants
   WHERE account_id = '00000000-0000-0000-0000-000000000099'),
  0,
  'Soft-deleted account restaurants not visible'
);

SELECT finish();
ROLLBACK;
```

10. Future Scaling Considerations

Scenario	Bottleneck	Mitigation in Current Design	Future Action Required
10K accounts, 50K restaurants	idx_accounts_owner_id, idx_restaurants_account_id	B-Tree indexes on both FK columns. Auth Hook query becomes O(log n) not O(n).	No schema change required
Noisy Neighbor (one account = 5000 orders/min)	Per-account rate limiting at Edge Function layer	RLS does not rate-limit. PgBouncer transaction-mode pooling prevents connection exhaustion.	Implement rate_limits table in Phase 6
Multi-region deployment	account.id determines shard/region at provisioning	Registry Shard pattern: account metadata stored centrally, operational data stored regionally. account_id FK chain remains consistent.	Requires migration to separate Supabase project per region for enterprise tenants
Account with 100+ restaurants	idx_restaurants_account_id partial index (active only)	Account-scoped SELECT returns restaurants via indexed account_id scan. RLS evaluated once per query via initPlan.	No schema change; OLAP queries on restaurants table must use materialized view
7-10 year audit log retention	BRIN index on audit log executed_at	Monthly RANGE partitions on audit.change_log. Old partitions detached and moved to cold storage without locking live table.	Implement audit schema with partitioned tables in Phase 2 schema migration

✅ **ARCHITECTURAL NON-NEGOTIABLE:** This design is forward-compatible with the Hybrid Isolation Model described in PVD Section 14. Standard tenants (1-100) run on shared schema with RLS isolation. Enterprise tenants (high-volume) can be migrated to dedicated Supabase projects without schema redesign — account_id is the shard key for that migration.

11. Final Declaration

This architecture represents the minimum viable and maximum safe implementation of the Business Account → Multiple Restaurants tenant layer for befoodi V2. Every decision documented herein is traceable to the PVD constitutional baseline.

Principle	Implemented As
Security is a Database Property	RLS on accounts, account_members, restaurants. Hard DELETE USING(false) on all tables.
Production is the Source of Truth	Migration uses IF NOT EXISTS and RAISE EXCEPTION guards. Idempotent on re-run.
Verification Beats Generation	pgTAP negative test suite proves isolation. 8 negative test cases provided.
Performance is a Constraint	initPlan caching on all RLS predicates. Partial indexes on all active-record filters. No Lookup Loop.
Discipline is the Moat	Two-migration pattern. DEFERRABLE FKs. Cascade trigger. Hard DELETE blocked. Schema changes gated by CI.

Conformance to this baseline is mandatory.

Deviations are architectural errors. Manual overrides are governance failures.

befoodi V2 — Phase-4 Locked Baseline — February 2026